Tutorial Problem - 02

1. You are given a small dataset containing user hand-movement values (x, y, z accelerometer readings) used for basic VR gesture recognition.

Task:

a) Create a feedforward neural network using MATLAB's Neural Network Toolbox.

b) Train the network using the dataset.

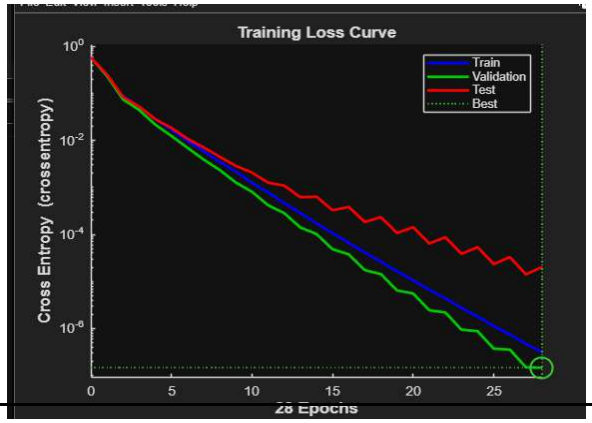
c) Plot the loss curve and comment on convergence

```
data = readtable('synthetic_vr_gesture_accelerometer.csv');

inputs  = [data.ax data.ay data.az]';   % 3 × N
targets = categorical(data.label);
targetsNN = dummyvar(targets)';

inputs = normalize(inputs, 2);

net = patternnet(10);   % one hidden layer with 10 neurons

net.trainFcn = 'trainscg';   % Scaled conjugate gradient
net.divideParam.trainRatio = 0.7;
net.divideParam.valRatio   = 0.15;
net.divideParam.testRatio  = 0.15;

[net, tr] = train(net, inputs, targetsNN);

outputs = net(inputs);
accuracy = sum(vec2ind(outputs) == vec2ind(targetsNN)) / length(targets);
disp(['Accuracy: ', num2str(accuracy*100), '%']);

figure;
plotperform(tr);
title('Training Loss Curve');
```

```
>> tutorial2a
Accuracy: 100%
```

The training loss decreases steadily over epochs, showing that the network is learning and converging. Small fluctuations at the start are normal, but the overall downward trend indicates stable training.

Output:



2. A convolutional layer receives an input image of size 256×256×3 and applies 32 filters of size 5×5, stride 1, and padding 2.

Task:

a) Compute the output feature map dimensions manually.

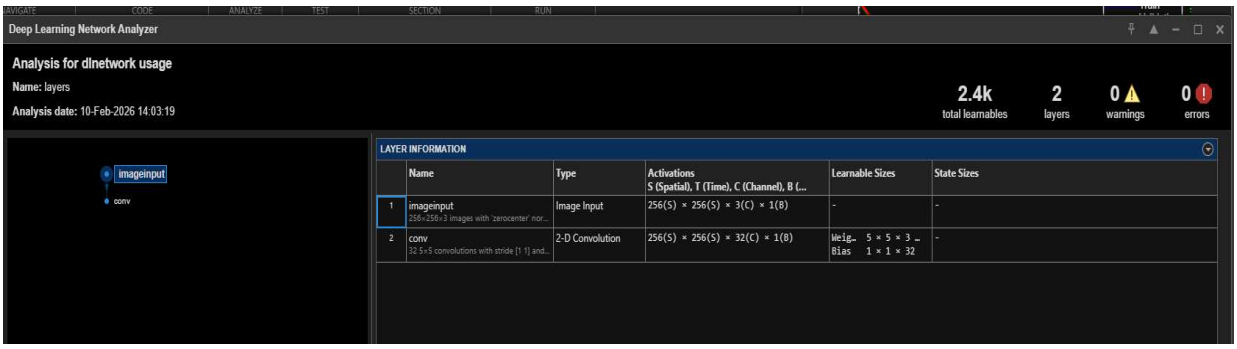
b) Verify your result by constructing the layer in MATLAB.

c) Explain how this feature map would help in XR hand recognition.

tutorial2a.m × untitled2.m ×

/MATLAB Drive/untitled2.m

```
inputSize = [256 256 3];
convLayer = convolution2dLayer( ...
    5, ...              % filter size
    32, ...             % number of filters
    'Stride', 1, ...
    'Padding', 2);


layers = [
    imageInputLayer(inputSize)
    convLayer
];
analyzeNetwork(layers);
```



The convolutional feature map plays a crucial role in XR hand recognition by extracting detailed spatial information about the hand while maintaining the original image resolution. This allows the system to detect subtle hand features and variations, providing a strong basis for accurate gesture interpretation in complex XR environments.

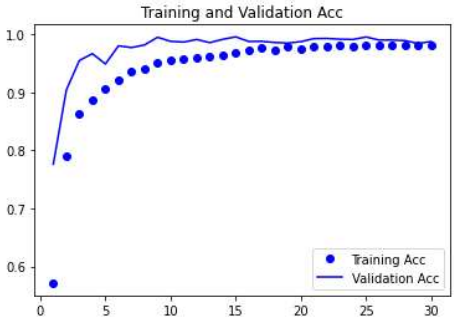
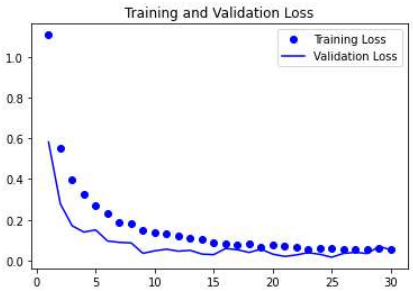
- Captures key hand features like edges, finger contours, and palm shapes through 32 distinct filters.

- Maintains full 256×256 resolution to preserve fine details such as finger positions.
- Enhances robustness to changes in lighting, hand orientation, and occlusion.
- Provides rich spatial information for deeper network layers to identify gestures and hand poses accurately.

Q3)A dataset contains images of 5 XR hand gestures captured using a VR camera.
Task:
a) Load a pretrained network (AlexNet/VGG16) in MATLAB.
b) Replace the last fully-connected and classification layers.
c) Train the model on your dataset using transfer learning.
d) Report training accuracy, validation accuracy, and show confusion matrix.

Code:

```
% Load the images from your directory
imds = imageDatastore('train', ...
'IncludeSubfolders', true, ...
'LabelSource', 'foldernames');
% Split data: 70% for training, 30% for validation
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.7, 'randomized');
% Load pretrained AlexNet
net = alexnet;
layers = net.Layers;
% Find the input size (AlexNet requires 227x227x3)
inputSize = layers(1).InputSize;
% Replace the last 3 layers
% Layer 23: Fully Connected, Layer 24: Softmax, Layer 25: Classification
numClasses = numel(categories(imdsTrain.Labels));
newLayers = [
fullyConnectedLayer(numClasses, 'Name', 'fc_new', 'WeightLearnRateFactor', 10,
'BiasLearnRateFactor', 10)
softmaxLayer('Name', 'softmax')
classificationLayer('Name', 'classoutput')
];
% Swap the old layers for the new ones
lgraph = layerGraph(layers);
lgraph = replaceLayer(lgraph, 'fc8', newLayers(1));
lgraph = replaceLayer(lgraph, 'prob', newLayers(2));
lgraph = replaceLayer(lgraph, 'output', newLayers(3));
% Resize images to 227x227
augImdsTrain = augmentedImageDatastore(inputSize(1:2), imdsTrain);
augImdsValidation = augmentedImageDatastore(inputSize(1:2), imdsValidation);
% Set training options
```

```
options = trainingOptions('sgdm', ...
'MiniBatchSize', 10, ...
'MaxEpochs', 6, ...
'InitialLearnRate', 1e-4, ...
'ValidationData', augImdsValidation, ...
'ValidationFrequency', 3, ...
'Verbose', false, ...
'Plots', 'training-progress');
% Train the network
trainedNet = trainNetwork(augImdsTrain, lgraph, options);
% Classify validation images
[YPred, scores] = classify(trainedNet, augImdsValidation);
YValidation = imdsValidation.Labels;
% Calculate Accuracy
accuracy = mean(YPred == YValidation);
fprintf('Validation Accuracy: %.2f%%\n', accuracy * 100);
% Show Confusion Matrix
figure
plotconfusion(YValidation, YPred)
title('Hand Gesture Recognition Confusion Matrix')
```

Output: